

1. Virtual Functions (Runtime Polymorphism)

A **virtual function** in C++ allows the most-derived function to be called when accessed via a base class pointer or reference. It enables **runtime polymorphism**, where the function that gets executed is determined at **runtime**, not compile-time.

Why Use Virtual Functions?

By default, function calls in C++ are resolved at compile time (called **early binding**). But in inheritance, when a derived class overrides a base class method, we often want to call the derived method even when using a base pointer. That's where `virtual` functions help.

Syntax:

```
class Base {
public:
    virtual void show() {
        std::cout << "Base class";
    }
};

class Derived : public Base {
public:
    void show() override {
        std::cout << "Derived class";
    }
};

int main() {
    Base* ptr = new Derived();
    ptr->show(); // Output: Derived class
}
```

Internal Working:

- C++ compiler creates a hidden **vtable** (virtual table).
- Each class with virtual functions has a pointer to this table.
- At runtime, the correct function is picked from the vtable.

Interview Questions:

- What is the purpose of virtual functions?
- How do virtual functions achieve runtime polymorphism?
- Why do we need virtual destructors?

2. Early Binding vs Late Binding

Early Binding

Function call is resolved at **compile-time**.

```
class A {  
public:  
    void print() {  
        std::cout << "A::print()";  
    }  
};
```

Late Binding (with virtual functions)

Function call is resolved at **runtime**.

```
class Animal {  
public:  
    virtual void sound() { std::cout << "Animal sound"; }  
};  
  
class Dog : public Animal {  
public:  
    void sound() override { std::cout << "Bark"; }  
};
```

3. Upcasting and Downcasting

Upcasting

Assigning a derived object to a base class pointer. It's safe and implicit.

```
Dog d;  
Animal* a = &d; // Upcasting
```

Downcasting

Casting base pointer to derived. Requires `dynamic_cast` and `virtual` functions.

```
Animal* a = new Dog();  
Dog* d = dynamic_cast<Dog*>(a);
```

If the cast fails, `dynamic_cast` returns `nullptr`.

4. Object Slicing

When a derived object is assigned to a base class **by value**, the extra properties from the derived class are sliced off.

```
class Base { public: int a; };
class Derived : public Base { public: int b; };
Base b = Derived(); // b.b is lost
```

5. Diamond Problem & Virtual Inheritance

Occurs in multiple inheritance:

```
class A { public: int x; };
class B : virtual public A {};
class C : virtual public A {};
class D : public B, public C {}; // Only one A instance
```

Use **virtual inheritance** to avoid multiple copies of base class A.

6. Shadowing

When a derived class re-declares a member already present in base, it **hides** it.

```
class Base { public: int x = 5; };
class Derived : public Base { public: int x = 10; };
```

You can access hidden member using `Base::x`.

7. Custom Exception Handling in C++

You can define your own exception types by inheriting from `std::exception`.

```
class MyException : public std::exception {
public:
    const char* what() const noexcept override {
        return "Custom exception occurred!";
    }
};
```

Use inside try-catch:

```
try {  
    throw MyException();  
} catch (const std::exception& e) {  
    std::cout << e.what();  
}
```

8. Name Mangling & extern "C"

Name Mangling: C++ modifies function names during compilation for function overloading support.

To prevent this and call C functions:

```
extern "C" void c_func();
```

9. Scope Resolution Operator `::`

Used to access:

- Global variables
- Class static members
- Base class methods

```
int x = 10;  
class A { public: static int x; };  
int A::x = 20;  
std::cout << ::x;    // global  
std::cout << A::x;   // class member
```

10. Pointer Concepts

Pointers store memory addresses.

```
int a = 10;  
int* p = &a;  
std::cout << *p;    // 10
```

Constant Pointers

```
int x = 5, y = 6;
const int* p1 = &x;    // can't modify *p1
int* const p2 = &x;    // can't change address
const int* const p3 = &x; // neither
```

Wild Pointer

Uninitialized pointer. Can cause bugs.

```
int* p; // wild pointer
```

Null Pointer

```
int* p = nullptr; // Safe
```

11. Friend Function (Detailed)

A friend function can access private members of a class.

Why Needed?

Sometimes, external functions need access to class internals (e.g., for operator overloading).

```
class Box {
private:
    int length;
public:
    Box(int l) : length(l) {}
    friend void printLength(Box);
};

void printLength(Box b) {
    std::cout << b.length;
}
```

12. STL (Standard Template Library) Overview

Provides **ready-to-use data structures** and algorithms.

Components:

- **Containers:** Hold data (vector, list, map, set, etc.)
 - **Algorithms:** Functions like sort, reverse, count
 - **Iterators:** Used to access container elements
 - **Functors:** Objects acting like functions
-

13. STL Vector Class (Detailed)

Dynamic array that resizes automatically.

```
std::vector<int> v = {1, 2, 3};  
v.push_back(4);
```

Important Methods:

- v.size(), v.capacity()
- v.front(), v.back()
- v.at(i), v[i]
- v.insert(), v.erase()
- v.clear(), v.empty()
- Iterators: begin(), end()

Performance:

Efficient random access, but insertion/deletion in middle is costly.

14. More STL Containers

list

Doubly linked list, fast insert/erase but no random access.

```
std::list<int> l = {1, 2, 3};  
l.push_back(4);
```

set

Stores unique sorted elements.

```
std::set<int> s = {4, 2, 1};
```

map

Key-value store with sorted keys.

```
std::map<std::string, int> m;  
m["age"] = 30;
```

stack / queue

Adapters for LIFO/FIFO structures.

```
std::stack<int> st;  
st.push(1);  
st.top();
```

Would you like to add topics like Templates, Operator Overloading, or Multithreading next?